TUNICARE

Plateforme intelligente de gestion de l’assurance santé

Cahier technique

Établissement : esprit

Classe : 4DAS4

Année universitaire : 2026–2027

Encadrant : Abd slem ghodhbeni

Membres de l'équipe :

- Safa Alouani
- Malek Alouani
- Islem sallemi
- Fatma abdlaziz
- Safa Baalouch

Table des matières

1. Introduction générale
2. Présentation générale du projet
3. Analyse fonctionnelle
4. Identification des acteurs
5. Architecture globale du système
6. Découpage en microservices
7. Conception détaillée des microservices
8. Communication et interactions entre les services
9. Conception des API REST
10. Sécurité et gestion des accès
11. Technologies et environnement technique
12. Gestion du projet et organisation de l'équipe
13. Planification prévisionnelle
14. Risques et solutions envisagées
15. Conclusion générale
16. Annexes

1. Introduction générale

1.1 Contexte

Le secteur de l'assurance santé connaît une transformation numérique importante. Les organismes d'assurance doivent gérer quotidiennement un grand nombre d'informations concernant les assurés, les contrats, les garanties, les professionnels de santé, les rendez-vous médicaux et les réclamations.

La gestion traditionnelle de ces informations peut entraîner plusieurs difficultés : dispersion des données, lenteur des traitements, manque de communication entre les différents acteurs et difficulté à suivre les demandes des assurés.

Dans ce contexte, la digitalisation des services d'assurance santé représente une solution permettant d'améliorer la qualité des services, de simplifier les procédures et de faciliter l'accès aux informations.

1.2 Problématique

Les systèmes classiques de gestion de l'assurance santé peuvent présenter plusieurs limites :

- Difficulté de centraliser les informations relatives aux assurés et à leurs garanties.
- Manque de visibilité sur les professionnels et établissements de santé partenaires.
- Complexité du suivi des réclamations.
- Difficulté à organiser et à suivre les rendez-vous médicaux.
- Dispersion des informations relatives aux dossiers médicaux.
- Manque de communication entre les différents modules du système.

La problématique principale est donc la suivante :

Comment concevoir une plateforme numérique centralisée, évolutive et sécurisée permettant de faciliter la gestion des services d'assurance santé et d'améliorer les interactions entre les différents acteurs ?

1.3 Solution proposée

Pour répondre à cette problématique, nous proposons TuniCare, une plateforme numérique dédiée à la gestion des services d'assurance santé.

La solution repose sur une architecture orientée microservices. Chaque microservice prend en charge un domaine fonctionnel précis et dispose de ses propres responsabilités et données.

La plateforme permet notamment de gérer :

- Les garanties et les couvertures d'assurance.
- Les professionnels et les établissements de santé.
- Les réclamations des assurés.
- Les rendez-vous médicaux.

- Les dossiers médicaux.

Une application frontend permet aux utilisateurs d'accéder aux fonctionnalités selon leurs rôles et leurs autorisations.

1.4 Objectifs du projet

Les principaux objectifs de TuniCare sont :

- Centraliser l'accès aux services liés à l'assurance santé.
- Simplifier la consultation des garanties.
- Faciliter la recherche des professionnels de santé.
- Permettre le dépôt et le suivi des réclamations.
- Organiser les rendez-vous médicaux.
- Faciliter la consultation des informations médicales autorisées.
- Améliorer la communication entre les différents services.
- Garantir la sécurité et la confidentialité des données.
- Concevoir une architecture évolutive et maintenable.

1.5 Périmètre du projet

Le projet couvre cinq domaines fonctionnels principaux :

Domaine	Description
Coverage	Gestion des garanties et des couvertures
Provider	Gestion des professionnels et établissements de santé
Reclamation	Gestion des réclamations
Appointment	Gestion des rendez-vous
Medical Record	Gestion des dossiers médicaux

Les fonctionnalités détaillées seront développées progressivement selon le planning du projet.

2. Présentation générale du projet

2.1 Fiche descriptive

Élément	Description
Nom du projet	TuniCare
Domaine	Assurance santé
Type d'application	Application web distribuée
Architecture	Microservices
Frontend proposé	Angular
Backend proposé	Java / Spring Boot
Communication	REST API
Base de données proposée	PostgreSQL
Gestion des conteneurs	Docker
Gestion du code	Git / GitHub

2.2 Description de la plateforme

TuniCare est une application web qui permet de regrouper plusieurs services liés à l'assurance santé dans une seule plateforme.

Chaque fonctionnalité est prise en charge par un microservice indépendant. Cette séparation permet de développer, tester et maintenir les différentes parties du système de manière indépendante.

L'application frontend communique avec les services backend à travers une passerelle d'API, appelée API Gateway.

2.3 Valeur ajoutée

- **Centralisation :** accès regroupé aux différents services.
- **Modularité :** chaque domaine fonctionnel est indépendant.
- **Évolutivité :** possibilité d'ajouter de nouvelles fonctionnalités.
- **Maintenabilité :** modification d'un service sans devoir reconstruire toute l'application.
- **Sécurité :** contrôle des accès et protection des informations sensibles.
- **Disponibilité :** possibilité de faire évoluer et de déployer les services séparément.

3. Analyse fonctionnelle

3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que la plateforme doit fournir à ses utilisateurs.

3.1.1 Gestion des garanties – Coverage Service

Le système doit permettre de :

- Consulter les garanties d'assurance.
- Afficher les types de couverture.
- Consulter les taux de remboursement.
- Afficher les plafonds de remboursement.
- Vérifier la période de validité d'une couverture.
- Consulter les prestations prises en charge.
- Consulter les informations relatives à un contrat ou à une formule d'assurance.

3.1.2 Gestion des professionnels de santé – Provider Service

Le système doit permettre de :

- Ajouter un professionnel de santé.
- Modifier les informations d'un professionnel.
- Supprimer ou désactiver un professionnel.
- Consulter la liste des professionnels.
- Rechercher un professionnel par nom ou spécialité.
- Gérer les spécialités médicales.
- Gérer les établissements de santé, notamment les cliniques et les pharmacies.

3.1.3 Gestion des réclamations – Reclamation Service

Le système doit permettre de :

- Déposer une nouvelle réclamation.
- Consulter les réclamations déposées.
- Afficher les détails d'une réclamation.
- Modifier le statut d'une réclamation selon les autorisations.
- Ajouter une réponse ou un commentaire.
- Suivre l'évolution du traitement.
- Consulter l'historique des réclamations.

3.1.4 Gestion des rendez-vous – Appointment Service

Le système doit permettre de :

- Consulter les disponibilités des professionnels de santé.
- Demander un rendez-vous.
- Consulter les rendez-vous.
- Modifier un rendez-vous.
- Annuler un rendez-vous.
- Consulter le statut d'un rendez-vous.
- Associer un rendez-vous à un professionnel de santé.

3.1.5 Gestion des dossiers médicaux – Medical Record Service

Le système doit permettre de :

- Créer un dossier médical.
- Consulter les informations médicales autorisées.
- Ajouter des informations relatives aux consultations.
- Enregistrer des documents médicaux.
- Consulter l'historique médical autorisé.
- Mettre à jour les informations selon les droits d'accès.

3.2 Besoins non fonctionnels

Besoin	Description
Sécurité	Protéger les données personnelles et médicales
Performance	Garantir des temps de réponse acceptables
Disponibilité	Assurer l'accès aux fonctionnalités
Évolutivité	Permettre l'ajout de nouveaux services
Maintenabilité	Faciliter la correction et l'évolution du code
Fiabilité	Garantir la cohérence des traitements
Ergonomie	Proposer une interface claire et facile à utiliser
Interopérabilité	Faciliter la communication entre les services

3.3 Contraintes du système

- Chaque microservice doit avoir une responsabilité fonctionnelle bien définie.

- Chaque microservice doit gérer ses propres données.
- Les communications entre services doivent passer par des interfaces clairement définies.
- Les accès aux données sensibles doivent être contrôlés.
- Les API doivent être documentées.
- Le code source doit être versionné avec Git.

4. Identification des acteurs

Les acteurs représentent les utilisateurs ou systèmes qui interagissent avec la plateforme.

4.1 Acteurs principaux

4.1.1 Administrateur

L'administrateur est responsable de la gestion globale de la plateforme. Ses principales responsabilités sont :

- Gérer les utilisateurs et leurs rôles.
- Administrer les paramètres généraux.
- Superviser les différents services.
- Contrôler les accès.
- Consulter les informations de gestion autorisées.

4.1.2 Assuré

L'assuré est l'utilisateur qui bénéficie des services de la plateforme. Il peut :

- Consulter ses garanties.
- Rechercher des professionnels de santé.
- Demander des rendez-vous.
- Déposer des réclamations.
- Suivre ses demandes.
- Consulter les informations de son dossier médical auxquelles il est autorisé à accéder.

4.1.3 Professionnel de santé

Le professionnel de santé intervient dans la prise en charge médicale. Il peut, selon les autorisations définies :

- Consulter les rendez-vous qui le concernent.
- Gérer ses disponibilités.
- Consulter les informations nécessaires à la prise en charge.
- Ajouter des informations médicales autorisées.

4.1.4 Gestionnaire des réclamations

Cet acteur intervient dans le traitement des réclamations. Il peut :

- Consulter les réclamations reçues.
- Examiner les demandes.
- Modifier leur statut.
- Ajouter des réponses.
- Assurer leur suivi.

4.1.5 Gestionnaire de la plateforme

Cet acteur peut intervenir dans l'administration fonctionnelle des services selon les responsabilités attribuées.

Remarque : les rôles définitifs doivent être harmonisés avec les règles métier validées par l'équipe. Un acteur ne doit pas être considéré comme implémenté tant que ses permissions ne sont pas définies.

4.2 Matrice des acteurs et fonctionnalités

Fonctionnalité	Assuré	Administrateur	Professionnel	Gestionnaire
Consulter les garanties	Oui	Oui, selon droits	Non, sauf autorisation	Selon droits
Gérer les professionnels	Non	Oui	Non	Selon droits
Déposer une réclamation	Oui	Non	Non	Non
Traiter une réclamation	Consulter le suivi	Selon droits	Non	Oui
Demander un rendez-vous	Oui	Selon droits	Non	Non
Gérer les disponibilités	Non	Selon droits	Oui	Selon droits
Consulter un dossier médical	Selon droits	Non par défaut	Selon autorisation	Non par défaut

5. Architecture globale du système

5.1 Choix de l'architecture microservices

Pour la conception de TuniCare, nous proposons une architecture microservices.

Cette architecture consiste à diviser l'application en plusieurs services indépendants, chacun responsable d'un domaine fonctionnel.

Contrairement à une architecture monolithique où toutes les fonctionnalités sont regroupées dans une seule application backend, l'architecture microservices permet de séparer les responsabilités.

Chaque microservice possède :

- Une logique métier indépendante.
- Une API permettant l'accès à ses fonctionnalités.
- Une base de données dédiée.
- Un cycle de développement et de déploiement indépendant.

5.2 Composants de l'architecture

5.2.1 Frontend

Le frontend constitue l'interface utilisateur de TuniCare. Il est proposé de le développer avec Angular. Ses responsabilités sont :

- Afficher les différentes interfaces.
- Gérer les interactions avec les utilisateurs.
- Envoyer les requêtes HTTP.

- Afficher les réponses des services.
- Gérer les formulaires et les validations côté client.

5.2.2 API Gateway

L'API Gateway constitue le point d'entrée principal du backend. Il permet de :

- Recevoir les requêtes du frontend.
- Rediriger les requêtes vers les microservices correspondants.
- Centraliser certaines fonctions de sécurité.
- Appliquer des règles de routage.
- Éviter que le frontend connaisse directement l'adresse de chaque service.

5.2.3 Eureka Discovery Server

Eureka est proposé comme service de découverte. Il permet aux microservices de s'enregistrer auprès d'un registre central et de découvrir les autres services disponibles.

5.2.4 Config Server

Le Config Server permet de centraliser les configurations des différents microservices. Il facilite la gestion des paramètres communs et évite de dupliquer certaines configurations dans chaque service.

5.2.5 Microservices métier

La plateforme repose sur cinq microservices :

- Coverage Service.
- Provider Service.
- Reclamation Service.
- Appointment Service.
- Medical Record Service.

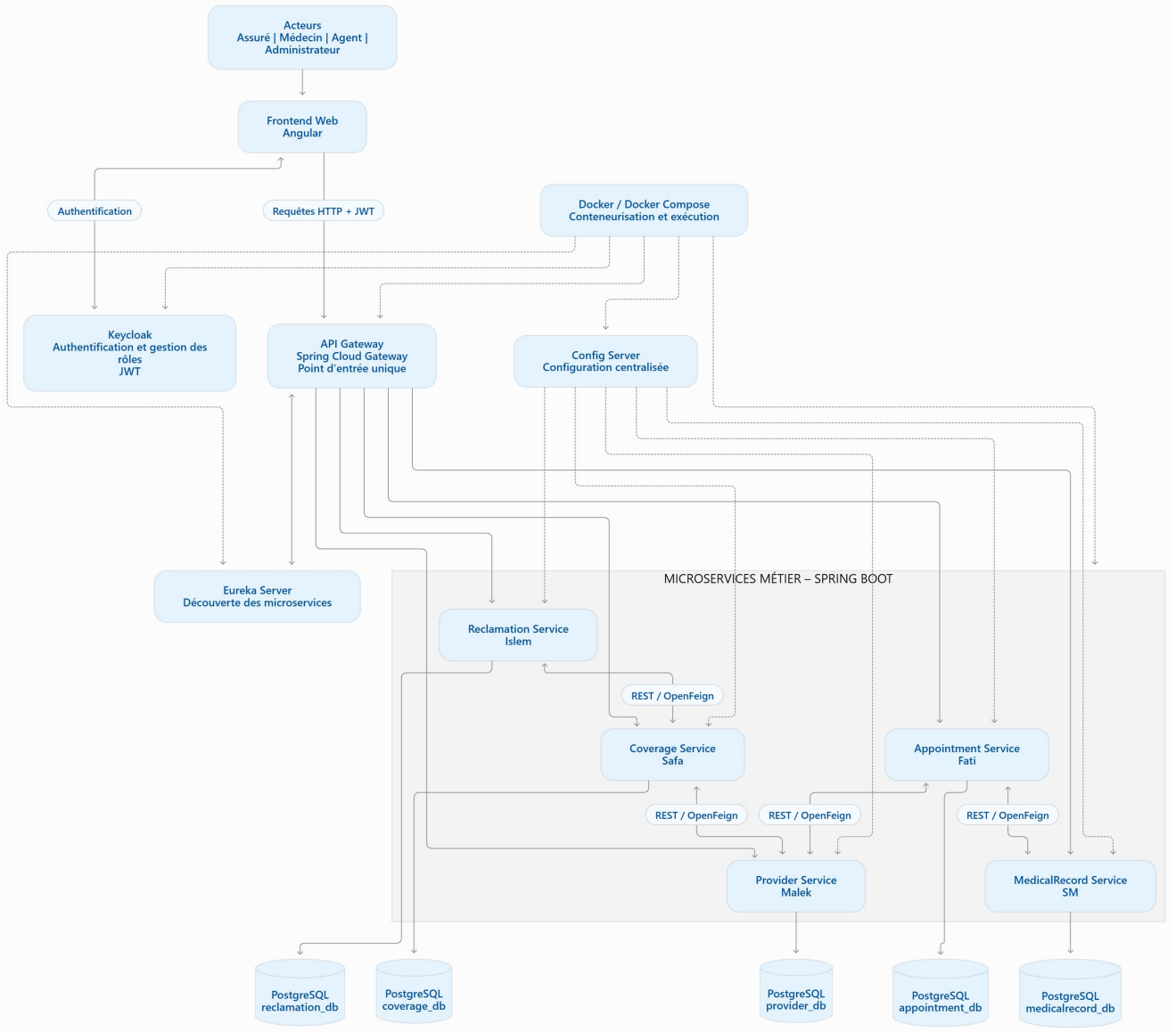
5.2.6 Bases de données

Chaque microservice possède sa propre base de données afin de garantir l'indépendance des données et de limiter le couplage entre les services.

5.2.7 Service de sécurité

Keycloak peut être utilisé pour gérer l'authentification et les rôles des utilisateurs avec des jetons JWT. Ce choix reste à confirmer avec l'équipe.

5.3 Diagramme d'architecture globale



5.4 Justification des choix architecturaux

Composant	Justification
Angular	Développement d'une interface web structurée et dynamique
Spring Boot	Développement de services backend indépendants
API Gateway	Centralisation des points d'entrée
Eureka	Découverte dynamique des services
Config Server	Centralisation de la configuration
PostgreSQL	Gestion des données relationnelles
Docker	Facilitation du déploiement
Keycloak	Gestion centralisée de l'authentification

6. Découpage en microservices

6.1 Principe du découpage

Le système est divisé en cinq microservices métier.

Chaque service correspond à un domaine fonctionnel spécifique. Ce découpage permet de limiter les dépendances et de faciliter le travail collaboratif entre les membres de l'équipe.

6.2 Tableau récapitulatif

Microservice	Responsabilité	Responsable
Coverage Service	Gestion des garanties	Safa Alouani
Provider Service	Gestion des professionnels de santé	Malek
Reclamation Service	Gestion des réclamations	Islem
Appointment Service	Gestion des rendez-vous	Fati
Medical Record Service	Gestion des dossiers médicaux	SM

6.3 Indépendance des microservices

Chaque microservice possède sa propre base de données.

Cette règle permet de :

- Limiter les dépendances entre les services.
- Éviter le partage direct des entités JPA.
- Permettre une évolution indépendante.
- Faciliter les tests.
- Réduire les risques de modification simultanée des mêmes données.

Les échanges de données entre microservices s'effectuent par des API ou des événements, et non par un accès direct à la base de données d'un autre service.

7. Conception détaillée des microservices

7.1 Coverage Service

7.1.1 Présentation

Le Coverage Service est responsable de la gestion des garanties et des couvertures d'assurance santé.

Il permet de définir les prestations prises en charge, les taux de remboursement, les plafonds et les périodes de validité.

7.1.2 Responsabilités

- Gérer les informations des couvertures.
- Gérer les types de garanties.
- Définir les taux de remboursement.

- Définir les plafonds annuels.
- Consulter la validité des couvertures.
- Fournir les informations nécessaires aux autres services autorisés.

7.1.3 Modèle de données proposé

Entité Coverage

Attribut	Type	Description
id	Long	Identifiant unique
name	String	Nom de la couverture
description	String	Description
coverageType	String	Type de couverture
startDate	Date	Date de début
endDate	Date	Date de fin
status	String	État de la couverture
annualLimit	Decimal	Plafond annuel

Entité Guarantee

Attribut	Type	Description
id	Long	Identifiant unique
name	String	Nom de la garantie
reimbursementRate	Decimal	Taux de remboursement
description	String	Description
coverageId	Long	Référence à la couverture

Exemples de garanties

Prestation	Taux fictif
Consultation	80 %
Analyses médicales	70 %
Radiologie	75 %
Médicaments	60 %

Ces valeurs sont des exemples destinés à illustrer le fonctionnement du service. Elles ne représentent pas des taux contractuels réels.

7.1.4 API proposées

Méthode	Endpoint	Description
GET	/api/coverages	Consulter toutes les couvertures
GET	/api/coverages/{id}	Consulter une couverture
POST	/api/coverages	Ajouter une couverture
PUT	/api/coverages/{id}	Modifier une couverture
DELETE	/api/coverages/{id}	Supprimer une couverture
GET	/api/coverages/{id}/guarantees	Consulter les garanties

7.1.5 Responsable

Safa Alouani – Coverage Service

7.2 Provider Service

7.2.1 Présentation

Le Provider Service est chargé de la gestion des professionnels et des établissements de santé.

Il permet de centraliser les informations relatives aux médecins, cliniques, pharmacies et spécialités médicales.

7.2.2 Responsabilités

- Gérer les professionnels de santé.
- Gérer les spécialités médicales.
- Gérer les cliniques.
- Gérer les pharmacies.
- Rechercher un professionnel.
- Consulter les informations d'un établissement.

7.2.3 Modèle de données proposé

Entité Provider

Attribut	Type	Description
id	Long	Identifiant unique
name	String	Nom du professionnel
email	String	Adresse électronique
phone	String	Numéro de téléphone
type	String	Type de professionnel
status	String	État du professionnel

Entité Specialty

Attribut	Type	Description
id	Long	Identifiant unique
name	String	Nom de la spécialité
description	String	Description

Entité Clinic

Attribut	Type	Description
id	Long	Identifiant unique
name	String	Nom de la clinique
phone	String	Téléphone
address	String	Adresse

Entité Pharmacy

Attribut	Type	Description
id	Long	Identifiant unique
name	String	Nom de la pharmacie
phone	String	Téléphone
address	String	Adresse

Les associations entre ces entités doivent être définies selon le modèle métier final.

7.2.4 API proposées

Méthode	Endpoint	Description
GET	/api/providers	Liste des professionnels
GET	/api/providers/{id}	Détails d'un professionnel
POST	/api/providers	Ajouter un professionnel
PUT	/api/providers/{id}	Modifier un professionnel
DELETE	/api/providers/{id}	Supprimer un professionnel
GET	/api/specialties	Liste des spécialités
GET	/api/clinics	Liste des cliniques
GET	/api/pharmacies	Liste des pharmacies

7.2.5 Responsable

Malek Alouani– Provider Service

7.3 Reclamation Service

7.3.1 Présentation

Le Reclamation Service permet de gérer les réclamations déposées par les assurés.

Il assure le suivi des demandes depuis leur création jusqu'à leur résolution.

7.3.2 Responsabilités

- Enregistrer une réclamation.
- Consulter les réclamations.
- Modifier les informations autorisées.
- Gérer les statuts.
- Ajouter des réponses.
- Conserver l'historique des traitements.

7.3.3 Modèle de données proposé

Entité Reclamation

Attribut	Type	Description
id	Long	Identifiant unique
subject	String	Objet de la réclamation

description	String	Description
submissionDate	DateTime	Date de dépôt
status	String	Statut
category	String	Catégorie
insuredId	Long	Identifiant de l'assuré

Entité ReclamationResponse

Attribut	Type	Description
id	Long	Identifiant unique
content	String	Contenu de la réponse
responseDate	DateTime	Date de réponse
reclamationId	Long	Référence de la réclamation
authorId	Long	Auteur de la réponse

7.3.4 Cycle de vie d'une réclamation

Une réclamation peut suivre les états suivants :

17. **Submitted :** la réclamation est déposée.
18. **In Progress :** la réclamation est en cours de traitement.
19. **Resolved :** une solution a été apportée.
20. **Rejected :** la réclamation a été rejetée.

Le passage d'un état à un autre doit respecter les règles de gestion définies par l'équipe.

7.3.5 API proposées

Méthode	Endpoint	Description
GET	/api/reclamations	Consulter les réclamations
GET	/api/reclamations/{id}	Consulter une réclamation
POST	/api/reclamations	Déposer une réclamation
PUT	/api/reclamations/{id}	Modifier une réclamation
PATCH	/api/reclamations/{id}/status	Modifier le statut
POST	/api/reclamations/{id}/responses	Ajouter une réponse
GET	/api/reclamations/{id}/history	Consulter l'historique

7.3.6 Responsable

Islem Sallemi – Reclamation Service

7.4 Appointment Service

7.4.1 Présentation

Le Appointment Service est responsable de la gestion des rendez-vous médicaux entre les assurés et les professionnels de santé.

Il permet d'organiser les demandes de rendez-vous et de suivre leur évolution.

7.4.2 Responsabilités

- Créer une demande de rendez-vous.
- Consulter les rendez-vous.
- Modifier un rendez-vous.
- Annuler un rendez-vous.
- Gérer les statuts.
- Associer les rendez-vous aux professionnels de santé.
- Vérifier les disponibilités selon les règles définies.

7.4.3 Modèle de données proposé

Entité Appointment

Attribut	Type	Description
id	Long	Identifiant unique
appointmentDate	DateTime	Date et heure
status	String	Statut
reason	String	Motif
insuredId	Long	Identifiant de l'assuré
providerId	Long	Identifiant du professionnel
createdAt	DateTime	Date de création

Les identifiants `insuredId` et `providerId` sont des références externes. Ils ne représentent pas des relations JPA directes vers les entités des autres microservices.

7.4.4 Cycle de vie d'un rendez-vous

Un rendez-vous peut avoir les statuts suivants :

- Pending : en attente de confirmation.
- Confirmed : confirmé.
- Cancelled : annulé.
- Completed : terminé.

7.4.5 API proposées

Méthode	Endpoint	Description
GET	/api/appointments	Consulter les rendez-vous
GET	/api/appointments/{id}	Consulter un rendez-vous
POST	/api/appointments	Créer un rendez-vous
PUT	/api/appointments/{id}	Modifier un rendez-vous
PATCH	/api/appointments/{id}/cancel	Annuler un rendez-vous
PATCH	/api/appointments/{id}/status	Modifier le statut

7.4.6 Responsable

Fatma abdlaziz– Appointment Service

7.5 Medical Record Service

7.5.1 Présentation

Le Medical Record Service est chargé de la gestion des dossiers médicaux des assurés.

Il permet de centraliser les informations médicales autorisées et de faciliter leur consultation par les personnes habilitées.

Ce service manipule des données particulièrement sensibles. Il doit donc appliquer des règles strictes de confidentialité et de contrôle d'accès.

7.5.2 Responsabilités

- Créer un dossier médical.
- Consulter un dossier médical autorisé.
- Ajouter des informations médicales.
- Gérer l'historique des consultations.
- Enregistrer les documents médicaux.
- Contrôler les autorisations d'accès.

7.5.3 Modèle de données proposé

Entité MedicalRecord

Attribut	Type	Description
id	Long	Identifiant unique
insuredId	Long	Identifiant de l'assuré
creationDate	DateTime	Date de création
medicalHistory	Text	Historique médical
notes	Text	Notes autorisées

Entité MedicalDocument

Attribut	Type	Description
id	Long	Identifiant unique
documentName	String	Nom du document
documentType	String	Type de document
uploadDate	DateTime	Date d'ajout
fileReference	String	Référence du fichier
medicalRecordId	Long	Référence du dossier médical

Entité MedicalConsultation

Attribut	Type	Description
id	Long	Identifiant unique

consultationDate	DateTime	Date de consultation
diagnosis	Text	Diagnostic
observations	Text	Observations
providerId	Long	Identifiant du professionnel
medicalRecordId	Long	Référence du dossier

Les champs définitifs doivent être validés selon le périmètre réel du projet et les règles de confidentialité retenues.

7.5.4 API proposées

Méthode	Endpoint	Description
GET	/api/medical-records/{id}	Consulter un dossier autorisé
POST	/api/medical-records	Créer un dossier
PUT	/api/medical-records/{id}	Mettre à jour un dossier
POST	/api/medical-records/{id}/documents	Ajouter un document
GET	/api/medical-records/{id}/documents	Consulter les documents autorisés
POST	/api/medical-records/{id}/consultations	Ajouter une consultation
GET	/api/medical-records/{id}/consultations	Consulter l'historique

7.5.5 Responsable

Safa Baalou– Medical Record Service

8. Communication et interactions entre les services

8.1 Principe général

Les microservices doivent communiquer afin de réaliser certains traitements nécessitant plusieurs domaines fonctionnels.

Dans TuniCare, les échanges peuvent être réalisés de deux manières :

- Communication synchrone par API REST.
- Communication asynchrone par événements, si un mécanisme de messagerie est retenu.

8.2 Communication synchrone

La communication synchrone permet à un service d'envoyer une requête à un autre service et d'attendre sa réponse.

Exemple : lorsqu'un assuré souhaite demander un rendez-vous, l'Appointment Service peut contacter le Provider Service pour vérifier les informations du professionnel concerné.

Le traitement peut être représenté ainsi :

21. L'assuré demande un rendez-vous depuis le frontend.
22. Le frontend envoie une requête à l'API Gateway.
23. L'API Gateway redirige la requête vers l'Appointment Service.

24. L'Appointment Service vérifie les informations nécessaires auprès du Provider Service.
25. Le rendez-vous est enregistré si les règles métier sont respectées.
26. Le résultat est renvoyé au frontend.

8.3 Communication asynchrone

La communication asynchrone permet à un service de publier un événement sans attendre que tous les autres services aient terminé leurs traitements.

Si l'équipe retient RabbitMQ, il pourrait être utilisé pour diffuser des événements tels que :

- AppointmentCreated.
- AppointmentCancelled.
- ReclamationSubmitted.
- ReclamationResolved.

Par exemple, lorsqu'une réclamation est créée, le Reclamation Service peut publier un événement `ReclamationSubmitted`. Un autre composant autorisé peut recevoir cet événement pour déclencher une notification.

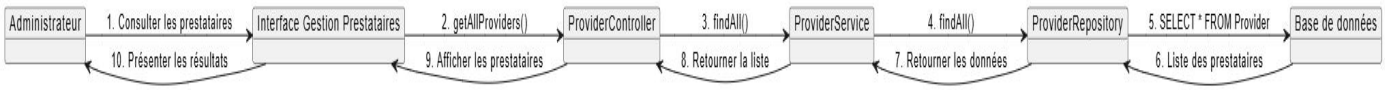
Attention : RabbitMQ est ici une proposition d'architecture. Il ne faut l'indiquer comme technologie utilisée que si son intégration est réellement prévue ou réalisée.

8.4 Tableau des interactions proposées

Service source	Service cible	Objectif
Appointment	Provider	Vérifier les informations d'un professionnel
Appointment	Coverage	Vérifier une couverture si nécessaire au processus
Reclamation	Coverage	Consulter les informations de garantie liées à une réclamation
Medical Record	Provider	Identifier le professionnel associé à une consultation
Coverage	Autres services	Fournir les informations de couverture autorisées

Les interactions définitives doivent être limitées aux besoins métier réels. Tous les services n'ont pas besoin de communiquer directement entre eux.

8.5 Diagramme de communication



9. Conception des API REST

9.1 Principe

Les API REST permettent aux applications et aux microservices de communiquer à travers le protocole HTTP.

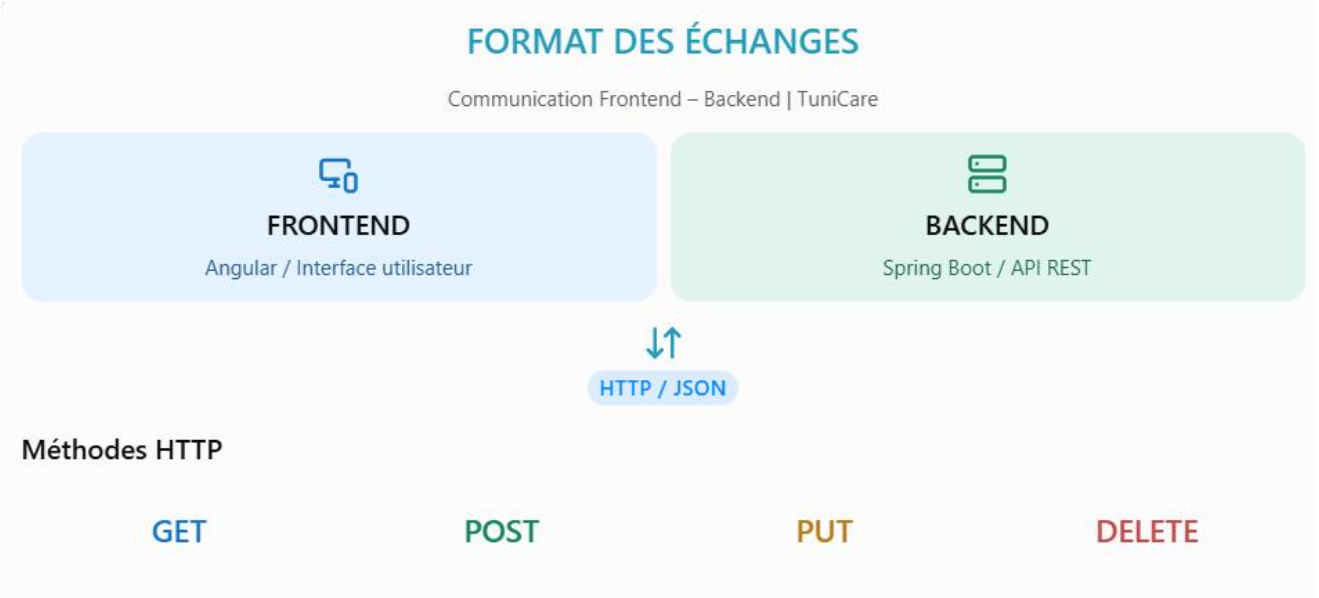
Chaque service expose des endpoints permettant d'accéder à ses fonctionnalités.

Les méthodes HTTP utilisées sont :

Méthode	Utilisation
GET	Récupérer des informations
POST	Créer une ressource
PUT	Modifier une ressource
PATCH	Modifier partiellement une ressource
DELETE	Supprimer une ressource

9.2 Format des échanges

Les échanges entre le frontend et le backend sont réalisés au format JSON.



Exemple de création d'une réclamation :

Exemple : Création d'une réclamation

REQUEST JSON

JSON

```
{
  "subject": "Remboursement non reçu",
  "category": "Reimbursement"
}
```

RESPONSE JSON

JSON

```
{
  "id": 1,
  "status": "SUBMITTED"
}
```

Ces exemples illustrent le format envisagé. Les noms de champs et les réponses définitives devront correspondre aux DTO réellement développés.

9.3 Codes de réponse HTTP

Code	Signification
200	Requête exécutée avec succès
201	Ressource créée
204	Opération réussie sans contenu retourné
400	Requête incorrecte
401	Utilisateur non authentifié
403	Accès interdit
404	Ressource introuvable
500	Erreur interne du serveur

9.4 Documentation des API

La documentation des endpoints pourra être réalisée avec Swagger / OpenAPI.

Elle permettra de :

- Présenter les endpoints.
- Décrire les paramètres.

- Afficher les formats JSON.
- Tester les requêtes.
- Faciliter le travail entre les membres de l'équipe.

10. Sécurité et gestion des accès

10.1 Objectifs de sécurité

La sécurité représente un aspect essentiel du projet TuniCare, notamment en raison de la sensibilité des informations médicales et personnelles.

Les objectifs sont :

- Garantir la confidentialité des informations.
- Empêcher les accès non autorisés.
- Vérifier l'identité des utilisateurs.
- Contrôler les permissions.
- Sécuriser les communications.
- Limiter l'exposition des données sensibles.

10.2 Authentification

L'authentification permet de vérifier l'identité de l'utilisateur avant de lui donner accès à la plateforme.

Une solution telle que Keycloak peut être utilisée pour gérer les comptes et les sessions.

10.3 Autorisation

L'autorisation consiste à déterminer les fonctionnalités accessibles à chaque utilisateur.

Un système basé sur les rôles peut être utilisé :

Rôle	Accès général
ADMIN	Administration de la plateforme
INSURED	Services destinés aux assurés
PROVIDER	Fonctionnalités destinées aux professionnels
CLAIM_MANAGER	Traitement des réclamations

Les permissions détaillées doivent être définies service par service.

10.4 Utilisation des JWT

Un JWT (JSON Web Token) est un jeton permettant de transmettre des informations d'authentification et d'autorisation.

Le fonctionnement envisagé est le suivant :

27. L'utilisateur saisit ses identifiants.
28. Le système d'authentification vérifie les informations.
29. Un jeton JWT est délivré.
30. Le frontend transmet le jeton lors des requêtes protégées.
31. L'API Gateway et/ou les microservices vérifient le jeton et les permissions.

32. L'accès est accordé ou refusé.

10.5 Protection des données médicales

Les données médicales nécessitent des protections renforcées :

- Accès limité aux personnes autorisées.
- Contrôle des permissions à chaque opération sensible.
- Journalisation des accès pertinents.
- Sécurisation des échanges par HTTPS en environnement de déploiement.
- Protection des fichiers et documents médicaux.
- Éviter l'exposition d'informations sensibles dans les logs.

11. Technologies et environnement technique

11.1 Technologies proposées

Technologie	Utilisation
Angular	Développement frontend
Java	Langage backend
Spring Boot	Développement des microservices
Spring Data JPA	Persistance des données
PostgreSQL	Gestion des bases de données
Spring Cloud Gateway	API Gateway
Eureka	Découverte des services
Spring Cloud Config	Configuration centralisée
Keycloak	Authentification et gestion des rôles
OpenFeign / WebClient	Communication REST entre services
RabbitMQ	Messagerie asynchrone, si retenue
Docker	Conteneurisation
Git	Gestion des versions
GitHub	Hébergement du code source
Swagger / OpenAPI	Documentation des API
Postman	Tests des API

11.2 Environnement de développement

Les membres de l'équipe peuvent utiliser :

- IntelliJ IDEA ou Eclipse pour le backend.
- Visual Studio Code pour le frontend.
- PostgreSQL et un outil d'administration de base de données.
- Postman pour tester les API.
- Git et GitHub pour la collaboration.
- Docker Desktop pour exécuter les conteneurs.

11.3 Organisation du backend

Structure proposée :

```
Backend/
├── api-gateway/
├── discovery-server/
├── config-server/
├── coverage-service/
├── provider-service/
├── reclamation-service/
├── appointment-service/
└── medical-record-service/
```

11.4 Structure interne d'un microservice

Exemple de structure du Coverage Service :

```
coverage-service/
├── src/
│   └── main/
│       ├── java/
│       │   └── com/tunicare/coverage/
│       │       ├── controller/
│       │       ├── service/
│       │       ├── repository/
│       │       ├── entity/
│       │       ├── dto/
│       │       ├── mapper/
│       │       ├── exception/
│       │       └── CoverageApplication.java
│       └── resources/
│           └── application.yml
├── pom.xml
└── README.md
```

Le nom du package Java doit être corrigé en respectant une syntaxe valide, par exemple `com.tunicare.coverage`.

11.5 Organisation du frontend

Structure proposée :

```
Frontend/
└── tunicare-frontend/
    ├── src/
    │   └── app/
    │       ├── core/
    │       ├── shared/
    │       ├── features/
    │       │   ├── coverage/
    │       │   ├── provider/
    │       │   ├── reclamation/
    │       │   ├── appointment/
    │       │   └── medical-record/
    │       └── app.routes.ts
    ├── angular.json
    └── package.json
```

12. Gestion du projet et organisation de l'équipe

12.1 Méthode de travail

L'équipe adopte une organisation collaborative inspirée de la méthode Agile.

Le travail est réparti en tâches indépendantes, avec des points réguliers pour vérifier l'avancement et résoudre les problèmes d'intégration.

Les tâches sont organisées autour de plusieurs étapes :

33. Analyse des besoins.
34. Conception de l'architecture.
35. Définition des modèles de données.
36. Développement des microservices.
37. Développement du frontend.
38. Intégration des services.
39. Tests et validation.
40. Documentation.

12.2 Répartition des responsabilités

Membre	Microservice	Responsabilités
Safa Alouani	Coverage	Garanties et couvertures
Malek	Provider	Professionnels et établissements
Islem	Reclamation	Réclamations
Fati	Appointment	Rendez-vous
SM	Medical Record	Dossiers médicaux

La mise en place des composants communs (Gateway, Discovery, Config Server, sécurité et déploiement) doit également être attribuée explicitement à un ou plusieurs membres.

12.3 Gestion de version avec Git

Git est utilisé pour assurer le suivi des modifications du code source.

GitHub permet de centraliser le dépôt et de faciliter la collaboration.

Chaque membre doit travailler sur une branche dédiée à sa fonctionnalité.

Exemple :

```
main
└── develop
    ├── feature/coverage-service
    ├── feature/provider-service
    ├── feature/reclamation-service
    ├── feature/appointment-service
    └── feature/medical-record-service
```

Les modifications doivent être intégrées dans la branche commune après vérification.

12.4 Règles de collaboration

- Chaque membre travaille sur son microservice.
- Les noms des branches doivent être explicites.
- Les commits doivent décrire clairement les modifications.
- Les modifications doivent être testées avant leur intégration.
- Les Pull Requests doivent être examinées.
- Les conflits Git doivent être résolus avant la fusion.
- Le dépôt doit contenir un README clair.
- L'enseignant doit être ajouté comme collaborateur du dépôt.

12.5 Organisation du dépôt GitHub

Structure générale recommandée :

```
TuniCare/
├── Backend/
│   ├── api-gateway/
│   ├── discovery-server/
│   ├── config-server/
│   ├── coverage-service/
│   ├── provider-service/
│   ├── reclamation-service/
│   ├── appointment-service/
│   └── medical-record-service/
├── Frontend/
│   └── tunicare-frontend/
├── Documentation/
│   ├── Presentation_Seance4.pdf
│   ├── Architecture_Globale.png
│   ├── Diagrammes/
│   └── Cahier_Technique_TuniCare.pdf
└── README.md
```

13. Planification prévisionnelle

Le projet peut être organisé selon les phases suivantes :

Phase	Activités	Résultat attendu
Phase 1	Analyse des besoins	Cahier des charges
Phase 2	Conception	Diagrammes et architecture
Phase 3	Mise en place du dépôt	Dépôt GitHub structuré
Phase 4	Développement backend	Microservices
Phase 5	Développement frontend	Interfaces utilisateur
Phase 6	Intégration	Communication entre services
Phase 7	Tests	Validation fonctionnelle
Phase 8	Documentation	Rapport et présentation

13.1 Suivi de l'avancement

Le suivi peut être assuré à travers GitHub Projects ou Trello.

Les tâches peuvent être réparties dans les catégories suivantes :

- À faire.
- En cours.
- En révision.
- Terminé.

Un suivi régulier permettra de détecter les retards et de réorganiser les priorités.

14. Risques et solutions envisagées

Risque	Conséquence possible	Solution envisagée
Mauvaise définition des interfaces	Difficultés d'intégration	Définir les contrats API à l'avance
Conflits Git	Perte de temps	Utiliser des branches indépendantes
Indisponibilité d'un service	Échec d'un traitement	Gérer les erreurs et les délais d'attente
Problèmes de communication	Échanges incorrects	Tester les API avec Postman
Données incohérentes	Erreurs métier	Définir les règles de validation
Accès non autorisés	Risque de fuite de données	Authentification et autorisation
Retard de développement	Livraison incomplète	Planification et suivi régulier
Mauvaise configuration	Échec du démarrage	Centraliser et documenter les configurations

14.1 Stratégie de test

Les tests prévus comprennent :

- Tests unitaires des différentes couches métier.
- Tests des endpoints REST.
- Tests de validation des données.
- Tests de communication entre microservices.
- Tests d'authentification et d'autorisation.
- Tests d'intégration.
- Tests fonctionnels des principaux scénarios.

Les outils pourront inclure JUnit, Mockito et Postman.

15. Conclusion générale

Le projet TuniCare vise à proposer une solution numérique destinée à faciliter la gestion des services d'assurance santé.

Grâce à une architecture basée sur les microservices, la plateforme sépare les responsabilités fonctionnelles en cinq domaines : les garanties, les professionnels de santé, les réclamations, les rendez-vous et les dossiers médicaux.

Cette organisation favorise la modularité, l'évolutivité et la maintenabilité du système.

Le présent cahier technique décrit les besoins fonctionnels et non fonctionnels, les acteurs, l'architecture globale, les responsabilités des microservices, les modèles de données, les API envisagées, les mécanismes de communication et les choix technologiques.

Cette conception constitue une base de référence pour les prochaines étapes de développement, d'intégration et de validation du projet.

16. Annexes

Annexe A – Liste des diagrammes à intégrer

Les diagrammes suivants sont recommandés pour compléter le cahier technique :

41. Diagramme de cas d'utilisation global.
42. Diagramme d'architecture globale.
43. Diagramme de composants.
44. Diagramme de déploiement.
45. Diagramme de classes du Coverage Service.
46. Diagramme de classes du Provider Service.
47. Diagramme de classes du Reclamation Service.
48. Diagramme de classes de l'Appointment Service.
49. Diagramme de classes du Medical Record Service.
50. Diagrammes de séquence des principaux scénarios.

Annexe B – Exemple de scénario : demande de rendez-vous

51. L'assuré se connecte à TuniCare.
52. Il consulte la liste des professionnels.
53. Il sélectionne un professionnel.
54. Il choisit une date et un créneau disponible.
55. Le frontend transmet la demande à l'API Gateway.
56. L'API Gateway redirige la requête vers l'Appointment Service.
57. L'Appointment Service vérifie les informations nécessaires.
58. Le rendez-vous est enregistré.
59. Le système retourne une confirmation à l'assuré.

Annexe C – Glossaire

Terme	Définition
Microservice	Service indépendant responsable d'un domaine fonctionnel
API	Interface permettant à des applications de communiquer
REST	Style d'architecture pour les services web
API Gateway	Point d'entrée central des requêtes

JWT	Jeton utilisé pour transmettre des informations d'authentification
Eureka	Service de découverte des microservices
Docker	Plateforme de conteneurisation
PostgreSQL	Système de gestion de base de données relationnelle
DTO	Objet utilisé pour transférer des données
CI/CD	Intégration continue et déploiement continu
OpenAPI	Spécification de documentation des API